# Week 8 Summary Report

## Project - E-Commerce Order Analytics System

### Objective

The goal for this week was to design and build a complete, end-to-end e-commerce order analytics system that ties together Python and SQL into a single working pipeline. Rather than working with a clean, ready-made dataset, the idea was to simulate what a real production environment usually looks like - messy, inconsistent, and full of small errors that need to be caught before any meaningful analysis can happen. The system was meant to cover the entire lifecycle of the data: starting from raw dataset generation, moving through cleaning and validation, loading into a proper relational database, and finally producing business-ready reports such as revenue breakdowns, customer rankings, retention trends, and segmentation. This made the task less about writing isolated scripts and more about thinking through how a data pipeline is actually structured and maintained end to end.

### Step 1: Generate Realistic Datasets

The first step involved writing a Python script to generate four core datasets - customers, products, orders, and order_items - using randomly generated and fake values so the data would resemble something a real e-commerce platform might produce. Instead of keeping the data clean from the start, several real-world inconsistencies were deliberately introduced into the generation process. These included NULL customer IDs, invalid or malformed email formats, inconsistent date formats across records, duplicate rows, orphan order items that referenced non-existent orders, negative quantities, and invalid discount values that fell outside a sensible range. The reasoning behind this was to make the cleaning stage in Step 2 meaningful rather than trivial, since a perfectly clean dataset wouldn't really test whether the validation logic actually worked. Once generated, all four datasets were exported as CSV files into a data/raw directory, which was kept separate from any cleaned output to preserve a clear boundary between raw and processed data.

### Step 2: Data Cleaning with Pandas

With the raw CSVs in place, the next step was to load them into Pandas DataFrames and work through the inconsistencies that had been intentionally introduced earlier. This involved handling missing values in a way that made sense for each column rather than blindly dropping rows, removing duplicate records, and standardizing product names so that inconsistent casing or spacing didn't create false distinctions between what should be the same product. Date formats were corrected and normalized to a single consistent format, since the raw data had a mix of styles that would have caused problems downstream in SQL. Email addresses were validated against a proper pattern to filter out clearly invalid entries, and referential integrity was checked across the datasets - for example, making sure order_items didn't reference orders or products

that didn't actually exist. Once all of this was done, the cleaned versions of each dataset were exported as new CSV files, ready to be loaded into the database in the next step.

## Step 3: Load into SQL Database

This step focused on moving the cleaned data out of flat CSV files and into a proper SQLite database with an actual schema behind it. Tables were created with Primary Keys and Foreign Keys to enforce the relationships between customers, products, orders, and order_items, along with NOT NULL and CHECK constraints to stop invalid data from being inserted in the first place. This was a useful exercise in thinking about schema design rather than just dumping data into tables, since the constraints had to reflect the actual business rules of the system - for instance, quantities and discounts needed sensible bounds, and foreign keys needed to correctly point back to valid parent records. After loading the cleaned datasets into their respective tables, the table relationships and row counts were verified manually to confirm that nothing had been lost or duplicated during the load process, and that the constraints were actually being enforced rather than silently ignored.

## Step 4: SQL Analytics - Joins & Aggregations

Once the data was sitting in a proper relational structure, this step was about actually extracting business value from it using SQL. JOIN queries were written to combine information across the customers, orders, order_items, and products tables, and aggregation functions were used on top of these joins to calculate metrics like total revenue by customer, by product category, and by month. Additional queries identified the top-selling products by revenue and computed the Average Order Value (AOV), which required carefully grouping orders and dividing total revenue by order count rather than by line item count. The revenue report shown below, covering the period from 2026-06-01 to 2026-06-30, summarizes total revenue broken down by category alongside the top 15 customer-category-month combinations by revenue, giving a quick view into which categories and customers were driving the most business during that period.

## Step 5: Window Functions & CTEs

This step pushed the SQL work a step further by introducing window functions and Common Table Expressions (CTEs) to handle analysis that plain GROUP BY queries can't really express cleanly. DENSE_RANK() was used to rank customers by spending and lifetime value without leaving gaps in the ranking when there were ties, while LAG() helped compare a customer's current period spending against their previous period to spot trends over time. NTILE() was used to split customers into buckets for segmentation purposes, and SUM() OVER() along with AVG() OVER() enabled running totals and moving averages to be calculated directly within a single query rather than requiring multiple passes over the data. CTEs made these queries far more readable by breaking complex logic into named, reusable steps, which also made debugging a lot easier when a particular ranking or running total didn't look right. The top

customers and rankings report below shows the top 10 customers by total spending alongside a DENSE_RANK() breakdown of the top 10 customers by lifetime value for the same period.

```
============================================
 REPORT: TOTAL REVENUE PER CATEGORY
============================================
+-------------+---------------+
|  category   | total_revenue |
+-------------+---------------+
| Electronics |     217113.24 |
| Home        |      91687.34 |
| Clothing    |      57197.49 |
| Books       |       18499.9 |
+-------------+---------------+
Total Rows: 4
```

```
=====================================================================
 REPORT: TOP 10 CUSTOMERS BY TOTAL ORDER VALUE
=====================================================================
+-------------+--------------------+---------------+-------------------+
| customer_id |   customer_name    | customer_type | total_order_value |
+-------------+--------------------+---------------+-------------------+
| CUST0083    | Mia Miller         | REGULAR       |          12182.19 |
| CUST0013    | Avery White        | REGULAR       |          11834.34 |
| CUST0044    | Kevin Ramirez      | VIP           |           9945.36 |
| CUST0062    | Isabella Hernandez | REGULAR       |           9635.32 |
| CUST0028    | Emily Davis        | REGULAR       |           9622.17 |
| CUST0018    | Ava Jones          | REGULAR       |           9582.04 |
| CUST0014    | Michael Williams   | REGULAR       |           9552.04 |
| CUST0004    | Jane Garcia        | REGULAR       |           9318.98 |
| CUST0086    | Christopher Allen  | REGULAR       |           9303.21 |
| CUST0025    | Isabella Scott     | REGULAR       |           8516.38 |
+-------------+--------------------+---------------+-------------------+
Total Rows: 10
```

## Step 6: Cohort & Retention Analysis

Cohort analysis was the focus of this step, and it involved grouping customers into cohorts based on two different reference points - their registration month and their first purchase month - to see how retention patterns differed depending on which milestone was used as the anchor. For each cohort, monthly retention

rates were calculated across the following few months (M0 through M3) to track what percentage of customers from that cohort were still active over time. This made it possible to identify both repeat customers who kept coming back and churned customers who dropped off after their initial interaction. Building this out in SQL required careful date-bucketing logic to correctly assign each customer to their cohort month and then join that against their subsequent order activity to compute the retention percentages. The cohort retention report below shows the top 10 cohorts by both registration month and first purchase month, along with their M1, M2, and M3 retention percentages, which turned out to vary quite a bit month to month rather than following a clean trend.

REPORT: COHORT RETENTION ANALYSIS (MONTHS 0-3)

| cohort_month | cohort_size | m0_active | m0_retention | m1_active | m1_retention | m2_active |
|---|---|---|---|---|---|---|
| 2024-01 | 4 | 0 | 0.0% | 0 | 0.0% | 2 |
| 2024-02 | 5 | 0 | 0.0% | 0 | 0.0% | 1 |
| 2024-03 | 7 | 0 | 0.0% | 0 | 0.0% | 0 |
| 2024-04 | 3 | 0 | 0.0% | 0 | 0.0% | 0 |
| 2024-05 | 4 | 1 | 25.0% | 1 | 25.0% | 1 |
| 2024-06 | 6 | 1 | 16.67% | 0 | 0.0% | 1 |
| 2024-07 | 4 | 1 | 25.0% | 1 | 25.0% | 0 |
| 2024-08 | 9 | 1 | 11.11% | 2 | 22.22% | 4 |
| 2024-09 | 6 | 1 | 16.67% | 1 | 16.67% | 1 |
| 2024-10 | 2 | 0 | 0.0% | 0 | 0.0% | 0 |
| 2024-11 | 5 | 0 | 0.0% | 0 | 0.0% | 0 |
| 2025-01 | 3 | 0 | 0.0% | 0 | 0.0% | 0 |
| 2025-02 | 4 | 0 | 0.0% | 1 | 25.0% | 0 |
| 2025-03 | 7 | 1 | 14.29% | 1 | 14.29% | 1 |
| 2025-04 | 5 | 0 | 0.0% | 3 | 60.0% | 3 |
| 2025-05 | 2 | 0 | 0.0% | 0 | 0.0% | 1 |
| 2025-06 | 5 | 0 | 0.0% | 4 | 80.0% | 0 |
| 2025-07 | 10 | 0 | 0.0% | 3 | 30.0% | 3 |
| 2025-08 | 7 | 1 | 14.29% | 2 | 28.57% | 0 |
| 2025-09 | 3 | 0 | 0.0% | 2 | 66.67% | 2 |
| 2025-10 | 3 | 0 | 0.0% | 1 | 33.33% | 0 |
| 2025-11 | 4 | 1 | 25.0% | 3 | 75.0% | 2 |
| 2025-12 | 2 | 0 | 0.0% | 2 | 100.0% | 1 |
| 2026-01 | 4 | 1 | 25.0% | 3 | 75.0% | 4 |
| 2026-02 | 4 | 0 | 0.0% | 2 | 50.0% | 3 |
| 2026-03 | 2 | 2 | 100.0% | 2 | 100.0% | 2 |

Total Rows: 26

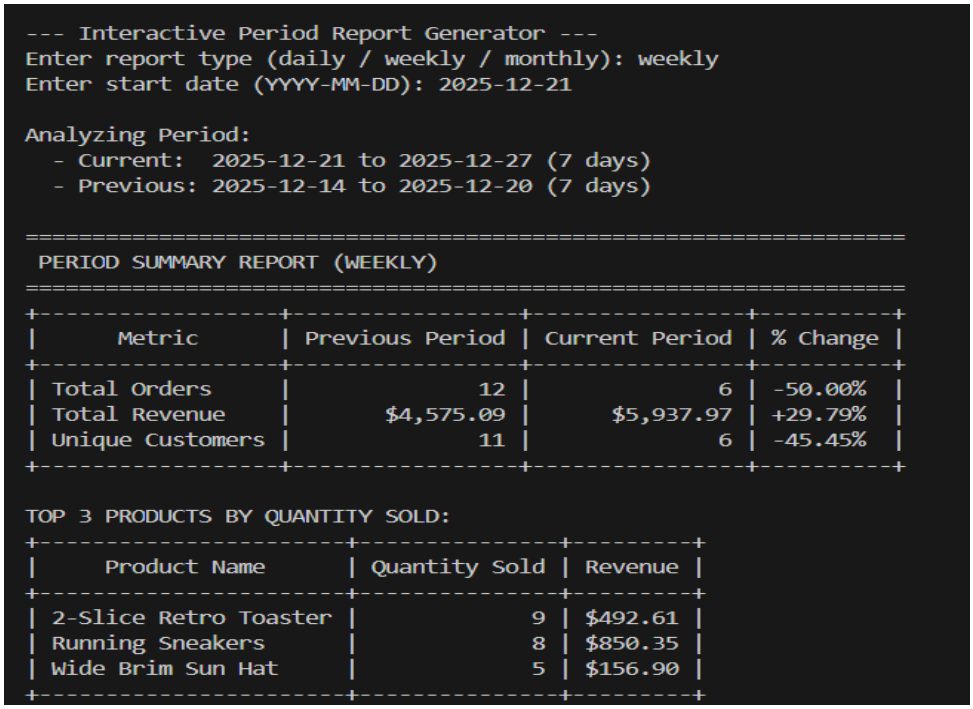
## Step 7: Customer Segmentation

Building on the retention work, this step segmented customers using a combination of purchase frequency, spending tiers, and a proper RFM (Recency, Frequency, Monetary) analysis. Customers were first grouped by how often they purchased - one-time, occasional, or loyal - and the Average Order Value was calculated for each of these frequency segments to see whether more frequent buyers were also spending more per order, which turned out not to be a straightforward relationship. The RFM analysis went a step further by scoring customers on how recently they purchased, how frequently they purchased, and how much they spent, then combining these scores into meaningful segments like Core Loyal, Active Engaged, At Risk Loyal, Regular, and Lost Customer. This gave a much richer picture than frequency alone, since it surfaced groups like At Risk Loyal customers who had high historical value but hadn't purchased recently, which is exactly the kind of segment a real business would want to target with re-engagement efforts. The

segmentation report below summarizes both the AOV by purchase-frequency segment and the full RFM segment breakdown with average recency, frequency, and spend for each group.

## Step 8: CLI Reporting Tool

To make all of this analysis actually usable without having to rewrite SQL queries every time, a command-line reporting tool called report_cli.py was built on top of the earlier SQL work. It supports multiple report types - daily, monthly, revenue, retention, and top_customers - which can each be run as a simple command with arguments for things like date range, without needing to touch the underlying SQL directly. The tool formats its output as clean console tables rather than raw query dumps, which makes it much easier to read at a glance and closer to something that could actually be handed to a non-technical stakeholder. The daily summary report shown below is a good example of the tool's output - it compares key metrics like total orders, net revenue, and unique customers against the previous period, lists the top products by revenue, and gives a day-by-day breakdown for the selected date range, all generated from a single CLI command.




## Step 9: Edge Case Handling

Since the datasets were deliberately built with messy, inconsistent data, a good part of this week went into making sure the whole pipeline could actually handle edge cases gracefully instead of just crashing or silently producing wrong numbers. This included validating orphan records that referenced missing parent rows, catching future-dated orders that shouldn't logically exist yet, flagging invalid discount values outside the expected range, and handling zero-quantity orders that would otherwise mess up revenue calculations. The reporting tool itself also needed to handle empty result sets cleanly - for example, when a date range had no matching orders - rather than throwing an error or displaying a blank, confusing table. On top of

that, invalid CLI inputs like bad date formats or unsupported report types needed to be caught with clear error messages instead of unhandled exceptions, which made the tool feel a lot more robust and closer to something production-ready rather than a one-off script.

## Step 10: Documentation & Submission

The final step was putting together proper documentation so that the project would actually make sense to someone opening the repository for the first time. A README was written covering the overall architecture of the pipeline, the folder structure separating raw data, cleaned data, and scripts, setup instructions for getting the project running locally, and step-by-step execution instructions for regenerating data, running the cleaning scripts, loading the database, and using the CLI reporting tool. Sample outputs were included directly in the documentation so that the reports could be understood without necessarily running the whole pipeline first. The GitHub repository structure was organized to reflect the different stages of the pipeline clearly, keeping raw datasets, cleaning scripts, database loading logic, SQL analytics queries, and the CLI tool in their own dedicated folders, which should make it easier to navigate and extend the project in the coming weeks.

## Conclusion

Overall, this week's work resulted in a genuinely complete end-to-end analytics pipeline covering everything from synthetic data generation with intentional real-world messiness, through cleaning and validation, into a properly constrained SQL database, and finally out to business-facing reports covering revenue, customer rankings, cohort retention, and RFM-based segmentation. Building the CLI reporting tool on top of all of this tied the project together into something that feels closer to an actual internal analytics tool than a set of disconnected scripts, and working through the edge cases forced a much more careful, defensive approach to the code than would have been needed with clean data. This project brought together data engineering, SQL analytics, and basic software design in a way that felt like a natural extension of what's been built in earlier weeks, and it gave a much clearer picture of how these pieces fit together in a real analytics workflow.